

PRE-REQUISITES

OSCAL Builder

Session Guide.

Everything you need to set up before the live session. Complete all steps below. If you can run the smoke test at the end, you are ready.

THE ONE-LINE PROMISE

Show up with the pre-reqs done, and you will build a working OSCAL compliance pipeline in the next 60 to 90 minutes.

WHAT YOU NEED

1. A fresh AWS free tier account (not your personal/work account)
2. IAM users configured with intentional security gaps
3. S3 buckets — one hardened, one intentionally misconfigured
4. CloudTrail enabled for audit logging
5. Python 3.10+ with boto3, openpyxl, requests
6. Prowler installed (open source cloud security scanner)
7. GitHub account with a personal access token
8. Docker Desktop with Jenkins running locally
9. The workshop starter repo cloned

ESTIMATED SETUP TIME

45 to 60 minutes. Do this the night before, not the morning of.

01 · AWS ACCOUNT

Create a fresh AWS account.

Do NOT use your personal or work AWS account. Create a brand new one at aws.amazon.com. Free tier covers everything we need. Use a new email address if needed (Gmail + aliases work: yourname+grclab@gmail.com).

02 · IAM USERS

Create 5 IAM users.

Each user is configured with intentional gaps so the pipeline has real findings to discover. Do not enable MFA on the test users — only on your admin user.

User	Access	Policy	MFA	Purpose
workshop-admin	Console	AdministratorAccess	YES	Your working account
demo-compliance	Console	ReadOnlyAccess	NO	Least privilege pass
demo-no-mfa	Console	PowerUserAccess	NO	MFA + privilege finding
demo-stale-key	CLI key	AdministratorAccess	NO	Key rotation + privilege
svc-pipeline	CLI key	SecurityAudit + ViewOnlyAccess	N/A	Pipeline service account

Save the svc-pipeline access key CSV — you will use this for boto3 and Prowler.

Configure your CLI after creating svc-pipeline:

```
aws configure --profile workshop
```

Enter the svc-pipeline access key ID and secret. Region: us-east-1. Output: json.

03 · S3 BUCKETS

Create 3 S3 buckets.

Add random characters to each name (S3 names are globally unique). All buckets in us-east-1.

BUCKET 1

workshop-encrypted-[random] — The hardened bucket

Default encryption: SSE-S3 (Amazon S3 managed keys)

Block all public access: ON (all four checkboxes checked)

Versioning: Enabled

This bucket passes SC-28 and SC-7 checks.

BUCKET 2

workshop-open-[random] — The misconfigured bucket

Encryption: Leave default SSE-S3 (AWS enforces this now)

Block all public access: OFF (uncheck all four boxes, acknowledge warning)

Versioning: Disabled

This bucket generates findings on public access and versioning.

BUCKET 3

workshop-logging-[random] — The broad access bucket

Block all public access: OFF (uncheck all four boxes)

Versioning: Disabled

After creation, go to Permissions > Bucket policy > Edit and paste:

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Sid": "BroadPublicRead",
    "Effect": "Allow",
    "Principal": "*",
    "Action": "s3:GetObject",
    "Resource": "arn:aws:s3:::YOUR-BUCKET-NAME/*"
```

```
}]  
}
```

Replace YOUR-BUCKET-NAME with your actual bucket 3 name.

04 · CLOUDTRAIL

Enable CloudTrail.

TRAIL

workshop-audit-trail

Go to CloudTrail in the AWS console. Click Create trail.

Trail name: workshop-audit-trail

Storage: Choose your encrypted bucket (bucket 1)

Log file SSE-KMS encryption: Disable (SSE-S3 from the bucket is enough)

Leave everything else default. Single region is fine.

This gives you a pass on AU-2 (Event Logging).

05 • TOOLS

Install Python tools.

Python 3.10 or higher required.

```
pip install boto3 openpyxl requests
pip install prowler
```

Prowler is an open source cloud security scanner. We use it as a free CSPM to generate findings against your AWS account — the same role that enterprise tools like Wiz or Qualys play in production environments.

06 • GITHUB

GitHub account + personal access token.

You need a GitHub account and a personal access token (PAT) with repo scope.

PAT**Create a personal access token**

Go to github.com > Settings > Developer settings > Personal access tokens > Tokens (classic)

Generate new token (classic). Name it 'grc-workshop'. Select the 'repo' scope.

Copy the token and save it — you will not see it again.

07 • JENKINS

Jenkins on Docker.

Install Docker Desktop if you do not have it. Then run:

```
docker run -d -p 8080:8080 --name jenkins jenkins/jenkins:lts-jdk21
```

Open <http://localhost:8080> in your browser. Get the initial admin password with:

```
docker exec jenkins cat /var/jenkins_home/secrets/initialAdminPassword
```

Complete the setup wizard. Install suggested plugins. Create one freestyle job called 'build-app' that runs 'echo build passed' as a shell step. Run it once.

08 • CLONETHEREPO

Clone the workshop starter repo.

```
git clone https://github.com/GRCJP/oscal-pipeline-workshop.git
cd oscal-pipeline-workshop
```

09 · S M O K E T E S T

Verify everything works.

YOU ARE READY IF ALL OF THESE PASS

```
python excel_to_oscal.py --help
```

Returns usage text with --input and --output flags

```
aws iam list-users --profile workshop
```

Returns JSON with your 5 IAM users

```
aws s3api list-buckets --profile workshop
```

Returns JSON with your 3 S3 buckets

```
curl -H 'Authorization: token YOUR_PAT' https://api.github.com/user
```

Returns your GitHub user profile JSON

```
curl http://localhost:8080/api/json
```

Returns Jenkins API response with your build-app job

```
prowler aws --profile workshop --list-checks | head -20
```

Returns a list of available Prowler checks

IF ANYTHING FAILS

Post in #live-sessions with the error and what step you are on. We will help you troubleshoot before the session, not during.

Stop scrolling. Start engineering.